LECTURE 2

Python Notes

Strings & Conditional Statements

Topics: Strings · Indexing · Slicing · String Functions · if-elif-else

1. Strings

String is a data type that stores a **sequence of characters**.

Creating Strings

```python
# Creating strings
name = "Apna College"
city = "Delhi"
empty = ""
```

Basic Operations

```python
# Concatenation — join strings with +
s1 = "hello"
s2 = "world"
print(s1 + s2) # helloworld
print(s1 + " " + s2) # hello world

# Length
name = "Apna College"
print(len(name)) # 12

# Repetition
print("ha" * 3) # hahaha
```

Note: Strings are **immutable** — you cannot change individual characters after creation.

2. Indexing

Each character has a **position (index)** starting at **0** from the left.

Positive Indexing

A	p	n	a	_	C	o	l	l	e	g	e
0	1	2	3	4	5	6	7	8	9	10	11

python
```python
str = "Apna_College"

print(str[0]) # A
print(str[1]) # p
print(str[5]) # C
print(str[11]) # e

# NOT allowed — strings are immutable!
str[0] = "B" # TypeError
```

Negative Indexing — count from the right

A	p	p	l	e
0	1	2	3	4
-5	-4	-3	-2	-1

python
```python
str = "Apple"

print(str[-1]) # e (last char)
print(str[-2]) # l
print(str[-5]) # A (first char)
```

Tip: str[-1] always gives the last character — easier than str[len(str)-1]!

3. Slicing

Extract a portion of a string. Syntax: **str[start : end]** — end is **NOT included**.

Basic Slicing

python

```python
str = "ApnaCollege"

str[1:4] # pna (indices 1, 2, 3)
str[0:4] # Apna
str[4:11] # College

# Omit start -> defaults to 0
str[:4] # Apna

# Omit end -> defaults to len(str)
str[4:] # College

# Copy entire string
str[:] # ApnaCollege
```

Negative Index Slicing

```python
str = "Apple"
# A p p l e
# -5 -4 -3 -2 -1

str[-3:-1] # pl
str[-3:] # ple
str[:-1] # Appl
```

Step Slicing (Bonus) — str[start : end : step]

```python
str = "ApnaCollege"

str[::2] # Every 2nd char
str[::-1] # Reverse: egelloCanpA
```

Tip: str[::-1] is the standard Pythonic way to reverse any string!

4. String Functions

Built-in methods to inspect and transform strings.

Method	Description	Returns
`str.upper()`	Converts all characters to uppercase	String

Method	Description	Returns
`str.lower()`	Converts all characters to lowercase	String
`str.capitalize()`	Capitalises only the first character	String
`str.endswith(sub)`	True if string ends with given substring	Bool
`str.find(word)`	Index of first occurrence (-1 if not found)	Integer
`str.replace(old, new)`	Replaces ALL occurrences of old with new	String
`str.count(sub)`	Counts how many times substring appears	Integer
`str.strip()`	Removes whitespace from both ends	String
`str.split(sep)`	Splits string into a list by separator	List

All Methods in Action

```python
str = "I am a coder."

# endswith
print(str.endswith("er.")) # True
print(str.endswith("xyz")) # False

# capitalize / upper / lower
print("hello".capitalize()) # Hello
print(str.upper()) # I AM A CODER.

# find
print(str.find("am")) # 2
print(str.find("xyz")) # -1

# replace
print(str.replace("coder", "developer")) # I am a developer.

# count
print(str.count("a")) # 2

# strip
print(" hello ".strip()) # hello

# split
print(str.split(" ")) # ['I', 'am', 'a', 'coder.']
```

5. Conditional Statements

Use **if – elif – else** to make decisions in your program.

Syntax

```python
if(condition):
Statement1 # runs if condition is True
elif(condition):
Statement2 # runs if above False, this is True
else:
StatementN # runs if ALL above were False
```

Note: Python uses **indentation** (4 spaces) to define code blocks — NOT curly braces!

Example 1 — Grade Calculator

```python
marks = int(input("Enter marks: "))

if marks >= 90:
grade = "A"
elif marks >= 80:
grade = "B"
elif marks >= 70:
grade = "C"
else:
grade = "D"

print("Grade:", grade)
# 95->A 83->B 72->C 55->D
```

Example 2 — Odd or Even

```python
n = int(input("Enter a number: "))

if n % 2 == 0:
print(n, "is Even")
else:
print(n, "is Odd")
```

Example 3 — Greatest of 3 Numbers

```python
a = int(input("1st number: "))
b = int(input("2nd number: "))
c = int(input("3rd number: "))

if a >= b and a >= c:
print("Greatest:", a)
elif b >= a and b >= c:
print("Greatest:", b)
else:
print("Greatest:", c)

# Short: print("Greatest:", max(a, b, c))
```

Example 4 — Multiple of 7

```python
n = int(input("Enter a number: "))

if n % 7 == 0:
print(n, "is a multiple of 7")
else:
print(n, "is NOT a multiple of 7")
```

6. Quick Reference

Topic	Syntax	Example
Concatenation	`s1 + s2`	`"hello"+"world" -> "helloworld"`
Length	`len(str)`	`len("Apple") -> 5`
Indexing	`str[i]`	`str[0] -> first char`
Neg. Index	`str[-i]`	`str[-1] -> last char`
Slicing	`str[s:e]`	`str[1:4] -> "pna"`
Reverse	`str[::-1]`	`"abc"[::-1] -> "cba"`
if / elif	`if cond: ...`	`if x>0: print("pos")`
endswith	`str.endswith(s)`	`returns True / False`
find	`str.find(word)`	`returns index or -1`
replace	`str.replace(o,n)`	`replaces all occurrences`

count	`str.count(sub)`	returns count of sub

7. Practice Problems + Solutions

Q1 Take user's first name as input and print its length.

solution

```
name = input("Enter first name: ")
print("Length:", len(name))
# e.g. name=Aman -> 4
```

Q2 Find the number of times '$' appears in a string.

solution

```
s = input("Enter string: ")
print(s.count("$"))
# e.g. a$b$c$ -> 3
```

Q3 Check if a number entered by the user is odd or even.

solution

```
n = int(input("Enter number: "))
if n % 2 == 0: print("Even")
else: print("Odd")
```

Q4 Find the greatest of 3 numbers entered by the user.

solution

```
a = int(input())
b = int(input())
c = int(input())
print(max(a, b, c))
```

Q5 Check if a number is a multiple of 7.

solution

```
n = int(input("Enter: "))
if n % 7 == 0:
print("Multiple of 7")
else: print("Not a multiple")
```